DESAFIO TÉCNICO | ESTÁGIO FULL STACK

To-Do List

Place Tecnologia e Inovação · Desafio para vaga de estágio Full Stack

Desenvolver uma aplicação web para gerenciamento de tarefas pessoais, com criação, consulta, edição, e exclusão, permitindo filtros de paginação, ordenação e pesquisa.

A entrega deverá ser realizada através de um link para um repositório público no github.

1. Tecnologias e infraestrutura

Camada	Tecnologia
Backend	Java 21+ e Spring Boot; API REST em JSON
Frontend	Angular 22 e Angular Material
Persistência	PostgreSQL 18 (acesso via JPA ou JDBC)
Ambiente	Docker compose para o frontend e backend
Diferenciais	Testes unitários, testes de comportamento, dockerfile, OpenAPI (Swagger) e Migrations via Flyway

2. Requisitos funcionais

- Cadastro: título obrigatório, descrição opcional, prioridade (LOW, MEDIUM e HIGH), status (PENDENT, ON_GOING, DONE) e vencimento opcional.
- Consulta: permitir visualizar os dados de uma tarefa individualmente.
- Edição: alterar título, descrição, prioridade, vencimento e status; permitir concluir e reabrir tarefas.
- Exclusão: remover tarefas, com tratamento quando o recurso não existir.
- Listagem: busca parcial por título, filtro por status e prioridade, ordenação por título, criação ou vencimento, e paginação executada no servidor. Os filtros devem ser combináveis.
- Dashboard: exibir o total de tarefas e totais por status

2.1. Identificação obrigatória por UUID

Cada tarefa terá um UUID gerado pelo backend como identificador. Utilizar java.util.UUID e o tipo nativo UUID no PostgreSQL 18. Todos os endpoints de acesso individual, atualização ou exclusão deverão utilizar esse identificador. O frontend deve consumir o UUID retornado; não é permitido definir ou alterar o ID na requisição.

UUID malformado: HTTP 400. UUID sintaticamente válido e inexistente: HTTP 404.

3. Backend, Onion Architecture e API REST

Aplicar os princípios da Onion Architecture: domínio independente de HTTP e JPA; camada de aplicação com casos de uso e regras; infraestrutura responsável por persistência e integrações; controllers responsáveis pela entrada e saída HTTP. A estrutura deve refletir as dependências entre as camadas, além dos nomes dos pacotes.

- Utilizar DTOs, validação de entrada e tratamento centralizado de exceções.
- Paginação, ordenação e filtros devem ser executados no PostgreSQL, não apenas sobre dados já carregados em memória.
- Utilizar métodos e códigos HTTP adequados; respostas com corpo seguem o contrato JSON abaixo.
- Evitar repetição desnecessária na montagem de respostas e não expor informações internas em falhas.

4. Contrato JSON obrigatório

Todas as respostas com corpo devem usar Content-Type: application/json e os campos abaixo. O código HTTP da resposta deve coincidir com code.

Campo	Tipo	Regra
status	string	Nome do status HTTP: OK, CREATED, NOT_FOUND...
code	int	Código HTTP real: 200, 201, 400, 404...
message	string	Mensagem descritiva da operação
payload	object ou null	Conteúdo retornado ou detalhes seguros do erro
timestamp	timestamp	ISO 8601, gerado pelo servidor
error	boolean	true para erro, false para sucesso

Exemplo:

```
{
    "status": "OK",
    "code": 200,
    "message": "success",
    "payload": {},
    "timestamp": "2026-10-29T13:30:00Z",
    "error": false
}
```

5. Requisitos do frontend

- Construir telas de listagem, formulário de criação/edição e dashboard de indicadores.
- Utilizar Angular Material, por exemplo MatTable, MatPaginator, MatSort, MatFormField, MatSelect, MatDialog e MatCard.
- Aplicar componentização, roteamento e Reactive Forms com validações.
- Separar lógica de apresentação e serviços de comunicação HTTP.
- Paginação, ordenação e filtros devem interagir com o backend; tratar corretamente os campos payload e error do contrato JSON.
- Exibir estados de carregamento, erro, ausência de dados e confirmação; oferecer responsividade mínima.

6. Docker e PostgreSQL

- Incluir Dockerfile para backend, Dockerfile para frontend e docker-compose.yml com Angular, Spring Boot e PostgreSQL 18.
- O comando docker compose up deve iniciar os serviços necessários.
- Configurar variáveis de ambiente, volume persistente para PostgreSQL e instruções de execução no README; não versionar segredos reais (adicionar .env.example).

7. Testes e diferenciais

São diferenciais: testes unitários com JUnit 5 e Mockito; cenários de comportamento executáveis com Cucumber/Gherkin; testes Angular e documentação da API utilizando OpenAPI (Swagger);

```
Funcionalidade: Gerenciamento de tarefas
  Cenário: Cadastrar tarefa com sucesso
    Dado que informo uma tarefa válida
    Quando envio a solicitação de cadastro
    Então a API deve retornar HTTP 201
    E a resposta deve conter um UUID
```

```
E a tarefa deve estar persistida
```

8. Entrega e documentação

Entregar repositório Git no Github com o código-fonte, arquivo README contendo instruções para execução com Docker Compose, comandos de testes (se implementados), descrição da arquitetura, exemplos de chamadas da API, funcionalidades concluídas e limitações conhecidas.

A entrega deve ser realizada em até 72 horas após o recebimento do desafio.

Os diferenciais da seção 7 serão considerados adicionalmente na comparação entre entregas, sem constituir exigência obrigatória.

Boa sorte!